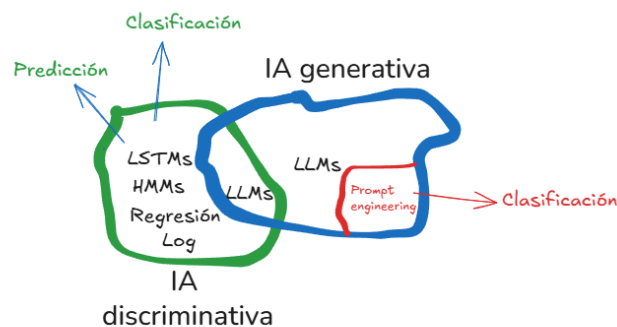


Taller 1: Procesamiento de datasets, tokenizadores y embeddings (seman. vectorial)

Introducción. La Inteligencia artificial se divide en dos áreas; una la *discriminativa* que propende por los procesos de clasificación y predicción con modelos clásicos matemáticos del aprendizaje supervisado de máquina y algunos LLMs + encoders-transformers como [BERT \(siglas de Bidirectional Encoder Representations from Transformers\)](#). La segunda área es la *generativa* basada en el modelo de *attention* + decoders-transformers que se soporta en estructuras matriciales y vectoriales de similitud a gran escala. La IA generativa se fundamenta exclusivamente de LLMs + decoders, los LLMs que se encuentran en la intersección de la figura hacen referencia a los modelos de LLMs + encoders como BERT que clasifican y predicen muy bien. De rojo se encuentra la parte de instrucciones para realizar *prompt engineering* para resolver tareas de clasificación y NER. En éste taller, en el punto 3 vamos a usar un clasificador basado en [BERT \(siglas de Bidirectional Encoder Representations from Transformers\)](#) para resolver un problema de análisis de sentimiento con tres clases (NUE:neutro, P:positivo, N:negativo). También en el punto 2 vamos a utilizar varios modelos de *sentences embeddings* basados en [BERT \(siglas de Bidirectional Encoder Representations from Transformers\)](#) y otros LLMs generativos.



1. Embeddings de palabras. Sobre el dataset [spanish billion words](#) entrenar los modelos de Word2vec y FastText sobre [Gensim](#) con el fin de obtener un modelo de embedding de palabras. Sobre los dos modelos entrenados se debe implementar los siguientes pasos:

1.1 Preprocesamiento del dataset

- Cargar el dataset [spanish billion words](#) luego preprocesarlo

- Tokenizar cada una de las sentencias del dataset y luego aplicar las siguientes transformaciones:
 - Eliminación de números.
 - Eliminación de signos de puntuación y símbolos especiales.
 - Eliminación de palabras vacías (*stopwords*) del español.
 - Tokenización de las sentencias en listas de palabras.
- *La idea es la de obtener una lista de listas de las palabras tokenizadas y limpias del dataset.*

1.2 Entrenamiento de modelos

Entrenar los siguientes modelos con las configuraciones indicadas:

Modelo	Tamaño de embeddings	Tamaño de sentencias
Word2vec	100, 200 y 300	500000, 5 y 10 Millones
FastText	100, 200 y 300	500000, 5 y 10 Millones

1.3 Consulta de similitud semántica

Implementar una función que dada una palabra de entrada, devuelva las 10 palabras más similares según el modelo seleccionado teniendo en cuenta el mejor modelo de los dos por sus respectivas características.

1.4 Visualización con t-SNE o con PCA

Visualizar en el plano cartesiano ([Tsne](#) o [PCA](#)) los vectores de embeddings entrenados con los modelos de [Wordvec](#) y [Fasttext](#) para números de sentencias de 50, 100 y 300. Comparar las visualizaciones de las palabras en el plano cartesiano, según los dos modelos del embedding de 300.

2. Tokenización con BETO para el Dataset BioBERT

Dataset : [BioBERT](#) (anotado, con características ["sentencia", "tag"]). Tener en cuenta que este datasets se encuentra de manera local.

Tareas :

2.1. Cargar los conjuntos **train**, **validation** y **test** usando `load_dataset("Biobert_json")`.

2.2. Mostrar :

- La sentencia `train[0]` con sus etiquetas `tag[0]`.
- La sentencia `validation[0]` con su etiqueta `tag[0]`.

2.3. Tokenizar cada una de las sentencias anteriores `train[0]` y `validation[0]` usando el tokenizador de BETO

```
from transformers import AutoTokenizer

tokenizer = AutoTokenizer.from_pretrained("dccuchile/bert-base-spanish-wwm-cased")
```

Ejemplo de uso

Abuela materna con cancer de mama a los 70 años .
B_FAMILY I_FAMILY O B_CANCER_CONCEPT I_CANCER_CONCEPT I_CANCER_CONCEPT O O B_QUANTITY B_METRIC O

Ejemplo de uso

```
['[CLS]',  
'Abuela',  
'materna',  
'con',  
'cancer',  
'de',  
'mama',  
'a',  
'los',  
'7',  
'##0',  
'años',  
'.',  
'[SEP]']
```

3. Embeddings de oraciones. Procesar documentos PDF, generar representaciones vectoriales de sus fragmentos mediante modelos de *sentence transformers* de [Hugging Face](#) y comparar el desempeño de distintos modelos en la recuperación semántica de texto mediante similitud coseno. Los siguientes son los pasos a seguir:

3.1 Carga y procesamiento de documentos PDF

- Leer todos los archivos en formato PDF ubicados en el directorio [pdf-coding](#)
- Utilizar [LangChain](#) junto con uno de los siguientes cargadores: [PyPDFLoader](#), o [pymupdf4llm](#) (recomendado para mejor preservación del layout y contenido).

3.2 Fragmentación del texto (*chunking*)

- Fragmentar cada documento en chunks de tamaño razonable que preserven coherencia semántica.
- La calidad del chunking impacta directamente en la efectividad de la recuperación; por tanto, se debe priorizar la integridad del significado dentro de cada fragmento

Aplicar el fragmentador:

```
python
1 text_splitter = RecursiveCharacterTextSplitter(
2     chunk_size=500,
3     chunk_overlap=50,
4     separators=["\n\n", "\n", ". ", " ", """]
5 )
```

3.3 Generación de embeddings de oraciones

- Utilizar la biblioteca **sentence-transformers** de [Hugging Face](#) para obtener representaciones vectoriales de cada fragmento.
- Evaluar los siguientes cuatro modelos, según sus características:

Modelo	Dimensiones	Uso provincial y eficiencia
all-mpnet-base-v2	768 (monolingüe)	Alta precisión-Media
all-MiniLM-L6-v2	384 (monolingüe)	General (rápido y eficiente)
paraphrase-multilingual-MiniLM-L12-v2	384 (multilingüe)	Búsqueda y similitud multilingüe
BAAI/bge-m3	1024 (multilingüe)	Recuperación y eficiencia, excelente para el español.

- Para cada modelo, generar las representaciones vectoriales de cada fragmento resultante del paso 2 anterior de chunking.

3.4 Consulta de similitud semántica

- Dada una oración de entrada (proporcionada por el usuario o definida en código), calcular su embedding para cada uno de los cuatro modelos.
- Comparar esta oración de entrada contra el fragmento más similar sobre cada modelo usando similitud coseno.
- Imprimir claramente:
 - El modelo utilizado.
 - El fragmento recuperado.

3.5 Visualización 2D con PCA (Ver ejemplo de `Todo_sobre_embeddings`)

- Para cada modelo, seleccionar:
 - La oración de entrada y el fragmento más similar según la similaridad del coseno
- Generar una gráfica en el plano cartesiano por modelo, mostrando:
 - La posición de la oración de entrada (marcada, por ejemplo, en rojo).
 - La posición del fragmento más similar (en azul o en verde).
- Incluir leyendas y títulos que identifiquen claramente el modelo utilizado.